

Task 1: Build a RESTful API with Spring Boot

1. Introduction

This document presents the implementation of a RESTful Product Management API developed using Spring Boot. The project demonstrates backend development concepts including REST architecture, CRUD operations, Spring Data JPA integration, DTO-based validation, and proper HTTP status code handling.

The application manages product information through REST endpoints and uses an H2 in-memory database for data persistence.

2. Objective

To develop a CRUD REST API for a Product resource using Spring Boot while implementing:

- Spring Web
 - Spring Data JPA
 - DTO-based request validation
 - Proper HTTP status codes
 - Layered architecture
 - H2 Database integration
-

3. Technologies Used

Technology	Purpose
Java 21	Programming Language
Spring Boot 3.5.x	Application Framework
Spring Web	REST API Development
Spring Data JPA	Database Operations
H2 Database	In-Memory Database
Maven	Dependency Management
Jakarta Validation	DTO Validation
Git & GitHub	Version Control

4. Project Architecture

The application follows a layered architecture:

Application Entry Point File:

ProductApiApplication.java **Code:**

```
1  package com.example.Product_api;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  @SpringBootApplication
7  public class ProductApiApplication {
8
9      public static void main(String[] args) {
10         SpringApplication.run(ProductApiApplication.class, args);
11     }
12
13 }
```

Responsibilities:

- Acts as the main entry point of the Spring Boot application.
- Contains the main() method used to start the application.
- Initializes the Spring Application Context.
- Enables component scanning to automatically detect Controllers, Services, Repositories, and other Spring-managed beans.
- Bootstraps the embedded Tomcat server and exposes REST endpoints on port **8080**.

Controller Layer

Handles incoming HTTP requests and sends appropriate responses.

File:

ProductController.java

Code:

```

1  package com.example.Product_api.controller;
2
3  import com.example.Product_api.dto.ProductDTO;
4  import com.example.Product_api.model.Product;
5  import com.example.Product_api.service.ProductService;
6  import jakarta.validation.Valid;
7  import org.springframework.http.HttpStatus;
8  import org.springframework.http.ResponseEntity;
9  import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/products")
15 public class ProductController {
16
17     private final ProductService service;
18
19     public ProductController(ProductService service) {
20         this.service = service;
21     }
22
23     // CREATE (POST)
24     @PostMapping
25     public ResponseEntity<Product> createProduct(@Valid @RequestBody ProductDTO productDTO) {
26         Product createdProduct = service.createProduct(productDTO);
27         return new ResponseEntity<>(createdProduct, HttpStatus.CREATED); // 201
28     }
29
30     // READ ALL (GET)
31     @GetMapping
32     public ResponseEntity<List<Product>> getAllProducts() {
33         return ResponseEntity.ok(service.getAllProducts()); // 200
34     }
35
36     // READ ONE (GET)
37     @GetMapping("/{id}")
38     public ResponseEntity<Product> getProductById(@PathVariable Long id) {
39         return service.getProductById(id)
40             .map(ResponseEntity::ok) // 200
41             .orElse(ResponseEntity.notFound().build()); // 404
42     }
43
44     // UPDATE (PUT)
45     @PutMapping("/{id}")
46     public ResponseEntity<Product> updateProduct(@PathVariable Long id, @Valid @RequestBody ProductDTO productDTO) {
47         return service.updateProduct(id, productDTO)
48             .map(ResponseEntity::ok) // 200
49             .orElse(ResponseEntity.notFound().build()); // 404
50     }
51
52     // DELETE
53     @DeleteMapping("/{id}")
54     public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
55         if (service.deleteProduct(id)) {
56             return ResponseEntity.noContent().build(); // 204
57         }
58         return ResponseEntity.notFound().build(); // 404
59     }
60 }

```

Responsibilities:

- Expose REST endpoints
- Accept request bodies
- Validate incoming data
- Return proper HTTP responses

Service Layer

Contains business logic.

File:

ProductService.java

Code:

```
1  package com.example.Product_api.service;
2
3  import com.example.Product_api.dto.ProductDTO;
4  import com.example.Product_api.model.Product;
5  import com.example.Product_api.repository.ProductRepository;
6  import org.springframework.stereotype.Service;
7
8  import java.util.List;
9  import java.util.Optional;
10
11 import org.slf4j.Logger;
12 import org.slf4j.LoggerFactory;
13
14 @Service
15 public class ProductService {
16
17     private static final Logger logger =
18         LoggerFactory.getLogger(ProductService.class);
19
20     private final ProductRepository repository;
21
22     public ProductService(ProductRepository repository) {
23         this.repository = repository;
24     }
25
26     // CREATE
27     public Product createProduct(ProductDTO dto) {
28
29         logger.info("Creating product: {}", dto.getName());
30
31         Product product =
32             new Product(
33                 dto.getName(),
34                 dto.getDescription(),
35                 dto.getPrice());
36
37         Product savedProduct = repository.save(product);
38
39         logger.info("Product created successfully with ID: {}", savedProduct.getId());
40     }
41 }
```

```

40
41     return savedProduct;
42 }
43
44 // READ ALL
45 public List<Product> getAllProducts() {
46
47     logger.info("Fetching all products");
48
49     return repository.findAll();
50 }
51
52 // READ ONE
53 public Optional<Product> getProductById(Long id) {
54
55     logger.info("Fetching product with ID: {}", id);
56
57     return repository.findById(id);
58 }
59
60 // UPDATE
61 public Optional<Product> updateProduct(Long id, ProductDTO dto) {
62
63     logger.info("Updating product with ID: {}", id);
64
65     return repository.findById(id)
66         .map(existingProduct -> {
67
68             existingProduct.setName(dto.getName());
69             existingProduct.setDescription(dto.getDescription());
70             existingProduct.setPrice(dto.getPrice());
71
72             logger.info("Product updated successfully: {}", id);
73
74             return repository.save(existingProduct);
75         });
76 }
77
78 // DELETE
79 public boolean deleteProduct(Long id) {
80
81     logger.info("Deleting product with ID: {}", id);
82
83     if (repository.existsById(id)) {
84
85         repository.deleteById(id);
86
87         logger.info("Product deleted successfully: {}", id);
88
89         return true;
90     }
91
92     logger.warn("Delete failed. Product not found: {}", id);
93
94     return false;
95 }
96 }

```

Responsibilities:

- Create products
- Fetch products
- Update products
- Delete products
- Coordinate between controller and repository

Repository Layer

Handles database operations.

File:

ProductRepository.java

Code:

```
1 package com.example.Product_api.repository;
2
3 import com.example.Product_api.model.Product;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface ProductRepository extends JpaRepository<Product, Long> {
9 }
```

Responsibilities:

- Communicate with H2 Database
- Perform CRUD operations using Spring Data JPA

Model Layer

Represents database entities.

File:

Product.java

Code:

```

1  package com.example.Product_api.model;
2
3  import jakarta.persistence.*;
4
5  @Entity
6  public class Product {
7      @Id
8      @GeneratedValue(strategy = GenerationType.IDENTITY)
9      private Long id;
10     private String name;
11     private String description;
12     private Double price;
13
14     // Default Constructor
15     public Product() {}
16
17     public Product(String name, String description, Double price) {
18         this.name = name;
19         this.description = description;
20         this.price = price;
21     }
22
23     // Getters and Setters
24     public Long getId() { return id; }
25     public void setId(Long id) { this.id = id; }
26     public String getName() { return name; }
27     public void setName(String name) { this.name = name; }
28     public String getDescription() { return description; }
29     public void setDescription(String description) { this.description = description; }
30     public Double getPrice() { return price; }
31     public void setPrice(Double price) { this.price = price; }
32 }

```

Fields:

- id
- name
- description
- price

The entity is mapped to the PRODUCT table using JPA annotations.

DTO Layer

Used for validation and request transfer.

File:

ProductDTO.java

Code:

```

1 package com.example.Product_api.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import jakarta.validation.constraints.Positive;
6
7 public class ProductDTO {
8
9     @NotBlank(message = "Product name cannot be blank")
10    private String name;
11
12    private String description;
13
14    @NotNull(message = "Price is required")
15    @Positive(message = "Price must be greater than zero")
16    private Double price;
17
18    // Default Constructor
19    public ProductDTO() {}
20
21    public ProductDTO(String name, String description, Double price) {
22        this.name = name;
23        this.description = description;
24        this.price = price;
25    }
26
27    // Getters and Setters
28    public String getName() { return name; }
29    public void setName(String name) { this.name = name; }
30    public String getDescription() { return description; }
31    public void setDescription(String description) { this.description = description; }
32    public Double getPrice() { return price; }
33    public void setPrice(Double price) { this.price = price; }
34 }

```

Validation Implemented:

Product Name Validation

@NotBlank

Ensures that the product name cannot be empty.

Price Validation

@NotNull

@Positive

Ensures:

- Price is provided
 - Price is greater than zero
-

5. Requirement Mapping and Implementation

1: Spring Web

Implementation

Added Spring Web dependency:

spring-boot-starter-web

Outcome

REST endpoints were successfully exposed through HTTP.

2: Spring Data JPA

Implementation

Added Spring Data JPA dependency:

spring-boot-starter-data-jpa

Created:

ProductRepository extends JpaRepository

Outcome

Database operations were performed without writing SQL queries manually.

3: CRUD Endpoints

Create Product

Endpoint: POST

/api/products

HTTP Status: 201

CREATED

Functionality:

- Creates a new product
 - Stores it in database
-

Get All Products

Endpoint:

GET /api/products

HTTP Status:

200 OK

Functionality:

- Retrieves all products
-

Get Product By ID Endpoint:

GET /api/products/{id}

HTTP Status:

200 OK 404 NOT
FOUND

Functionality:

- Retrieves a single product by ID
-

Update Product

Endpoint:

PUT /api/products/{id}

HTTP Status:

200 OK 404 NOT
FOUND

Functionality:

- Updates existing product details
-

Delete Product

Endpoint:

DELETE /api/products/{id}

HTTP Status:

204 NO CONTENT

404 NOT FOUND

Functionality:

- Deletes product from database

4: DTOs and Basic Validation

Implementation

Created:

ProductDTO

Validation annotations:

@NotBlank

@NotNull

@Positive

Outcome

Invalid requests are automatically rejected before reaching business logic.

Example:

```
{  
  "name": "",  
  "price": -100  
}
```

Produces validation errors.

6. Database Integration

H2 Database Configured

in:

application.properties

Features:

- In-memory database
 - Fast development environment
 - Easy testing
-

7. Conclusion

The project successfully fulfills all Task 1 requirements by implementing a complete CRUD REST API using Spring Boot. The application uses Spring Web for REST endpoints, Spring Data JPA for persistence, DTO-based validation for input verification, and proper HTTP status codes for client communication. Flyway migrations were used for database version control, and the application was verified through cURL commands and browser-based testing. This implementation demonstrates fundamental backend development practices and RESTful API design principles.

Linkedin Task1 Post Link: [CRUD REST API - Task1](#)

Github Repo Link: [subhojit1707-crypto/InternSpark-Internship](#)